Лекция 4. Функции

Хронометраж пары (90 мин): шаблон параметров ~5 мин → имя и подпись ~10 мин → область видимости ~20 мин → `lambda` ~20 мин → `map` ~20 мин → `filter` и при остатке `zip` ~15 мин.

Цели занятия

После лекции вы умеете:

1. Прочитать шаблон параметров: обязательный, со значением по умолчанию, `*args`, `**kwargs`.
2. Дать функции имя-глагол и подпись с типами. Знать, что Python эту подпись не проверяет.
3. Нарисовать рамку локальных имён. Отличить чтение внешнего имени, присваивание внутри функции, `global` и `nonlocal`.
4. Проследить `lambda`, `map` и `filter` на коротком списке.
5. Прочитать `reduce` в конце конспекта самостоятельно.

---

0. Зачем это в курсе

На семинаре 6 функция уже появилась: у неё есть имя, параметры и результат. `word_count` считает слова в строке, и тот же подсчёт вызывается для второй строки без копии цикла.

На этой лекции к уже написанной функции добавляется контракт и судьба имён. Контракт — что функция принимает и что возвращает. Судьба имён — какие имена живут только внутри вызова, а какие видны снаружи.

---

1. Шаблон параметров

Общий вид определения:

```
def имя_функции(параметр1, параметр2=значение, *args, **kwargs):
      ...
```

`параметр1` обязателен: вызов без него не состоится. `параметр2` можно не передавать, тогда берётся значение по умолчанию. `*args` собирает лишние аргументы по позиции. `**kwargs` записывает лишние аргументы по имени.
Два вызова одного и того же шаблона.

`имя_функции(1)` задаёт только обязательный параметр. У второго берётся значение по умолчанию. `*args` и `**kwargs` пустые. `имя_функции(1, 2, 3, доп=4)` раскладывается так: `параметр1` равен 1, `параметр2` равен 2, `3` попадает в `*args`, `доп` попадает в `**kwargs`.

---

2. Имя и подпись

Имя функции — глагол. По имени видно действие. Например, `word_count`: функция считает слова.

Подпись с типами показывает, что входит и что выходит:

```
def word_count(text: str) -> int:
      ...
```

В `text` входит строка. Наружу выходит целое число. Python эту подпись не проверяет. Неверная подпись программу не останавливает: она нужна читающему.

---

3. Область видимости

Вызов создаёт рамку локальных имён. На `return` рамка исчезает.

```
def add(a, b):
      return a + b


  add(1, 2)
```

Пока выполняется `add(1, 2)`, в рамке лежат `a = 1` и `b = 2`. Результат вызова равен 3. После `return` имён `a` и `b` снаружи нет.

Дальше четыре случая. В трёх первых снаружи есть `x = 2`.

Чтение внешнего имени

Функция может прочитать внешнее имя, если внутри нет своего присваивания в это имя.

```
x = 2

  def read_x():
      return x

  read_x()  # 2
```

Присваивание внутри

Присваивание внутри функции создаёт локальное имя. Внешнее имя остаётся прежним.

```
x = 2

  def write_local():
      x = 3
      return x

  write_local()  # 3
  x              # 2
```

`global`

`global` пишет в имя модуля. После вызова внешнее `x` меняется.

```
x = 2

  def set_global():
      global x
      x = 3

  set_global()
  x  # 3
```

`nonlocal`

`nonlocal` пишет в имя внешней функции. Здесь одна функция `inner` внутри `outer`. Второй вложенности нет.

```
def outer():
      n = 0
      def inner():
          nonlocal n
          n = n + 1
          return n
      return inner() + inner()

  outer()  # 3
```

`inner` вызывается дважды. Оба раза она увеличивает `n` функции `outer`. Сумма двух результатов равна 3.

---

4. `lambda`

`lambda` — функция из одного выражения. Слева от двоеточия параметры, справа результат. Имени у такой функции может не быть. Её можно положить в переменную.

```
square = lambda x: x * x
  square(3)  # 9
```

`square(3)` подставляет 3 вместо `x` и вычисляет `x * x`.

---

5. `map`

`map` применяет функцию к каждому элементу. Результаты он отдаёт по одному. `list` собирает их в список.

```
list(map(square, [1, 2, 3]))  # [1, 4, 9]
```

След по шагам: `square(1)` даёт 1, `square(2)` даёт 4, `square(3)` даёт 9.

---

6. `filter` и `zip`

`filter` оставляет те элементы, для которых функция вернула истину.

```
list(filter(lambda x: x > 0, [3, -1, 0, 4]))  # [3, 4]
```

`3 > 0` и `4 > 0` истинны. `-1 > 0` и `0 > 0` ложны, эти элементы в список не входят. `zip` собирает пары из двух списков и останавливается на более коротком.

```
list(zip([1, 2], ['a', 'b']))  # [(1, 'a'), (2, 'b')]
```

`reduce` это свёртка списка в одно значение. Сумму на семинаре пишут циклом. Разбор `reduce` лежит в следующем разделе.

---

7. К функциональному программированию

`map`, `filter` и `reduce` передают функцию туда, где иначе пишут цикл. Это заготовка к функциональному программированию.

```
from functools import reduce

  reduce(lambda x, y: x + y, [1, 2, 3, 4, 5])  # ((((1 + 2) + 3) + 4) + 5) = 15
  reduce(lambda x, y: x + y, [1, 2, 3], 0)     # 6
  reduce(lambda x, y: x + y, [], 0)            # 0
  reduce(lambda x, y: x + y, [10])             # 10
```

Функция внутри `reduce` принимает два аргумента: накопленное значение и следующий элемент. Счёт идёт слева направо. Начальное значение, если оно передано, стоит перед элементами. Пустой список без начального значения даёт `TypeError`. Список из одного элемента без начального значения возвращает этот элемент.

Самостоятельно

Сначала документация.

- Определение функций
- Аргументы, значения по умолчанию, `*args` и `**kwargs`
- `lambda`
- `map`
- `filter`
- `zip`
- `reduce`
- `string.punctuation`
- Обзор `map`, `filter` и `reduce`

Если документация сухая, те же темы по-русски.

- Яндекс. Учебник, 4.1. Функции и области видимости
- Яндекс. Учебник, 4.2. Аргументы, `map`, `filter`, `lambda`

- Metanit: функции
- Metanit: параметры
- Metanit: `return`
- Metanit: функция как параметр
- Metanit: `lambda`
- Metanit: область видимости
- pythontutor.ru, функции